Airport Simulator — Performance & Delivery Report

Oct 21, 2026 · Adrian Barra Cornes

Summary

The simulator runs a full 24-hour day with 2,857 flights in 6.6 ms, and the TreeMap time-stamp search is about 1,950x faster than a linear scan at 1 million rows. Speed and search figures come from new JUnit benchmarks run on 21 Oct 2026. Coverage comes from JaCoCo, which was added to the Maven build.

Metric	Result
Simulation speed	1 day, 2,857 flights in 6.6 ms (220,000 ticks/s); 30 days, 43,192 flights in 101 ms (426,000 ticks/s)
Range query	1M rows: 4.7 µs (TreeMap) vs 9.1 ms (linear), ~1,950x; 5M rows: ~7,700x
Test suite	73 tests, 100% passing; 79% line and 84% instruction coverage of the simulation model
Delivery	5 one-week sprints; 25 of 26 sprint tasks delivered; all 7 client-specification sections fully met

Test machine: AMD Ryzen 7 5800X, 32 GB RAM, Windows 11, JDK 25.0.2. All timings exclude JavaFX rendering.

Simulation speed

A simulated day (1,440 one-minute ticks) takes 3.5–6.6 ms for 478–2,857 flights, so speed stays above 200,000 ticks per second. Cost grows with flight count, not run length: a 30-day run takes 101 ms.

Days	Flow (flights/h each way)	Flights	Ticks	Median time (ms)	Ticks/s
1	10	478	1,440	3.71	388,000
1	15 (default)	715	1,440	3.46	416,000
1	20	953	1,440	4.30	335,000
1	30	1,430	1,440	3.93	366,000
1	60	2,857	1,440	6.55	220,000
7	30	10,068	10,080	19.15	526,000
30	30	43,192	43,200	101.32	426,000

How it was measured (`SimulationSpeedBenchmark`):

- Each run covers the whole pipeline: `DataGen` schedule generation, the tick loop and `ModelOutput.finishCalculations()`.
- The airport has 5 runways: 2 landing, 2 take-off, 1 mixed.
- Each configuration runs 3 warm-up passes, then 7 timed runs. The table shows the median.
- The benchmark copies the `SimLoop.call()` loop so the flow rate can change. The unmodified `SimLoop` was also timed at the default 15/h: 716 flights in 1.65 ms.

- `DataGen` spaces flights 60 ÷ rate minutes apart using integer division. Only rates that divide 60 give different schedules, so 45/h produces the same flights as 60/h.

Range-query performance

The TreeMap search stays at 2.5–8.3 µs from 1,000 rows up to 5 million rows. A linear scan grows in proportion to table size and reaches 64 ms at 5 million rows. The TreeMap is faster once a table has about 10,000 rows. Below that size, a scan of a small list is cheaper than walking the tree.

Rows	TreeMap (µs/query)	Linear scan (µs/query)	Speed-up
1,000	4.9	1.6	0.3x (linear faster)
10,000	2.5	23.2	9x
100,000	5.5	240.9	44x
1,000,000	4.7	9,146.5	~1,950x
5,000,000	8.3	63,674.7	~7,700x

How it was measured (`RangeQueryBenchmark`):

- The TreeMap side calls the code the tables actually use, `Globals.getArrivalsTableDataRange()` (`TreeMap.subMap`). This includes building the `ObservableList` the table displays.
- The linear side holds the same rows in an `ArrayList` and filters on the tick.
- Rows are spread 2 per tick, about one arrival and one departure per simulated minute. Each query asks for a one-hour (60-tick) window at a seeded random start, so it returns about 120 rows.
- Each run uses 500 warm-up queries, then reports the median of 2,000 TreeMap queries. The linear scan uses 2,000 queries, reduced to 20 million ÷ rows at the largest sizes. A check confirms both methods return the same rows.

Test suite

All 73 tests pass: 51 unit, 13 integration and 9 invariant tests. JaCoCo shows 79% line coverage of the simulation model. Coverage is 27% across the whole codebase because the JavaFX UI controllers have no automated tests.

Package	Lines	Instructions	Branches	Methods
`model` (simulation engine)	79.3% (758/956)	84.2%	78.7%	76.5%
`controller` (`SimLoop` , `SimService`)	59.2% (29/49)	59.4%	50.0%	28.6%
`UI` (JavaFX controllers)	0% (0/1,935)	0%	0%	0%
Whole project	26.6% (787/2,954)	28.9%	30.1%	31.2%

The best-covered classes are `DataGen` at 99%, `RunWay` at 97%, and `TakeOffQueue` and `RunWayMap` at 92–93%. The gaps are `HandleOutput` and `SimService` (0%), `Utils` (54%) and `AirCraft` (58%).

Suite	Tests
`RunWayTest`	16
`HoldingQueueTest`	13
`SimLoopScenarioTest` (integration)	13
`InvariantTest`	9
`MapsTest`	9
`TakeOffQueueTest`	9
`UtilsTest`	4

The build now includes `jacoco-maven-plugin` 0.8.14, which supports Java 25. Surefire is pinned to 3.5.2 because the cached 3.2.5 plugin failed to load. The two benchmarks are tagged `benchmark` and excluded from a normal `mvn test` run, so the suite still reports 73 tests.

Delivery

The team ran 5 one-week Scrum sprints and delivered 25 of 26 planned sprint tasks; the one dropped task was 2D graphics. All 7 sections of the client specification were fully met. Code cannot measure these figures, so they come from the sprint tables (Tables 6–8) and requirement tables (Tables 1 and 4) in the CS261 individual report, plus the git history.

Sprint	Tasks done that sprint	Notes
1	4 of 7	Take-off queue, navigation and input page carried over
2	6 of 7	`TrafficController` carried over
3	3 of 10	Most UI and testing work carried over; save/load statistics done
4	11 of 12	All carried-over work finished; 2D graphics discarded
5	1 of 1	Full re-test after bug fixes

- **Unique tasks:** 26 tasks were planned across the 5 sprints, counting each carried-over task once. 25 were delivered, or 96%.
- **Client specification:** 7 of 7 sections were fully met. These sections are arrivals, departures, runway mode, input parameters, information, user controls and output.
- **Team requirements document:** 7 of 10 sections were met as written. The other 3 were met with documented changes: input/data generation (1.1, 1.4), simulation information (2.3 not met) and runways (6.4 pre-scheduled instead of live).
- **Not delivered:** the live simulation view from the design document.
- **Repository:** 392 commits from 6 contributors between 16 Jan and 16 Oct 2026.

Method & reproducibility

Every figure above can be regenerated with two Maven commands run from `airport-app/`.

```
# 73 tests + JaCoCo report -> target/site/jacoco/index.html (and jacoco.csv)
mvn clean test

# benchmarks only -> target/benchmarks/simulation_speed.csv, range_query.csv, simloop_oneday.csv
mvn test -Dbenchmarks -Djacoco.skip=true
```

Files added or changed:

- `src/test/java/com/cs261/app/perf/SimulationSpeedBenchmark.java`
- `src/test/java/com/cs261/app/perf/RangeQueryBenchmark.java`
- `pom.xml`: JaCoCo plugin, Surefire 3.5.2, and a `benchmarks` profile that runs only `@Tag("benchmark")` tests

Caveats:

- `DataGen` uses an unseeded `Random`, so flight counts vary by a few flights between runs. Medians reduce the noise but do not remove it.
- The benchmarks run with `-Djacoco.skip=true` because the coverage agent instruments the code and slows it down.
- Found while benchmarking: `SimLoop` builds `DataGen` with `Utils.flow`, a constant 15, not `Utils.arrivalFlow` and `Utils.departureFlow`. As a result, the flow rates typed into the input form are ignored in the real app.